

Proyecto de Inteligencia Artificial

Agente Autónomo para HEX

Guillermo Hughes Cardona

1. Objetivo del agente

El objetivo principal del agente fue desarrollar un jugador autónomo para HEX capaz de seleccionar jugadas válidas e inteligentes a partir del estado actual del tablero, buscando maximizar sus posibilidades de conectar sus lados objetivo antes que el oponente. Además, el agente debía ajustarse a la estructura exigida por la orientación del proyecto, implementándose en un archivo `solution.py` mediante una clase `SmartPlayer` heredada de `Player`, con la lógica principal contenida en el método `play(board)`.

Desde el punto de vista funcional, el agente debía cumplir tres propiedades fundamentales:

- analizar el tablero actual e identificar oportunidades de conexión favorables;
- responder ante amenazas inmediatas del adversario;
- decidir una jugada dentro de un tiempo de ejecución compatible con el entorno competitivo del proyecto.

En este contexto, el problema no consiste únicamente en jugar de forma válida, sino en combinar conocimiento estratégico del dominio con técnicas de inteligencia artificial que permitan tomar decisiones robustas frente a distintos estilos de oponentes.

2. Enfoque general de la solución

La solución adoptada combina dos niveles de razonamiento complementarios. En primer lugar, se incorporó conocimiento específico del dominio de HEX, especialmente relacionado con estructuras de conexión, bridges, amenazas inmediatas y control del eje de avance del jugador. Este nivel aporta capacidad táctica y una interpretación geométrica del tablero.

En segundo lugar, dicho conocimiento se integró dentro de un enfoque de búsqueda basado en *Monte Carlo Tree Search* (MCTS). De esta forma, el agente no depende exclusivamente de reglas fijas, sino que puede explorar continuaciones futuras de la partida y utilizar esa información para respaldar la selección de jugadas.

En términos generales, la estrategia final se apoya en dos ideas:

1. utilizar heurísticas del dominio para reconocer patrones estructurales importantes del juego;
2. emplear búsqueda MCTS para evaluar alternativas futuras y mejorar la calidad global de la decisión.

Este enfoque híbrido permite combinar solidez táctica con capacidad de exploración, lo cual resulta especialmente útil en un juego como HEX, donde la calidad de las conexiones y la planificación posicional tienen un papel central.

3. Diseño del agente heurístico base

El agente heurístico base, conservado en el código como `SmartPlayerHeuristic`, fue diseñado como una versión fundamentada en reglas estratégicas explícitas. Su objetivo fue capturar patrones importantes del juego y servir como referencia estructural para el desarrollo posterior del agente final.

La lógica del agente heurístico sigue una organización por capas. Primero, verifica si existe una victoria inmediata disponible o si es necesario bloquear una victoria inmediata del oponente. Posteriormente, incorpora una regla específica para defender bridges propios cuando una de sus casillas críticas de soporte es ocupada por el rival. Luego intenta bloquear bridges rivales potenciales y construir bridges propios orientados de forma favorable respecto al eje ganador del jugador. Finalmente, cuando no se activa ninguna de las capas anteriores, utiliza una evaluación de respaldo basada en progreso posicional, presión sobre rutas de conexión y estructura local del tablero.

El elemento más importante de esta versión heurística es la representación geométrica de los bridges y sus supports. A partir de dicha representación, el agente puede reconocer conexiones fuertes entre fichas no adyacentes, valorar su orientación con respecto al objetivo del jugador y tomar decisiones tácticas asociadas a su construcción, protección o bloqueo. De esta manera, el agente heurístico establece una base estratégica interpretable y coherente con la naturaleza estructural del juego.

4. Diseño del agente MCTS final

El agente final del proyecto se basa en *Monte Carlo Tree Search* y está implementado a través de `SmartPlayerMCTS`, mientras que la clase pública `SmartPlayer` hereda de esta implementación para representar la versión definitiva de entrega. Este diseño permite mantener una separación clara entre la lógica heurística de apoyo y el agente final utilizado en competición.

Antes de ejecutar la búsqueda MCTS, el agente conserva un conjunto de tácticas duras consideradas fundamentales: detección de victoria inmediata, bloqueo inmediato y defensa de bridges propios comprometidos. Si ninguna de estas condiciones se cumple, el agente activa el proceso de búsqueda.

La búsqueda sigue las fases clásicas de MCTS:

- **Selección:** se desciende por el árbol utilizando una política basada en UCT para balancear exploración y explotación.
- **Expansión:** se genera un nuevo nodo a partir de una jugada aún no explorada.
- **Simulación:** se realiza un rollout guiado por heurísticas ligeras, evitando una simulación completamente aleatoria.
- **Retropropagación:** el resultado obtenido se propaga hacia atrás para actualizar las estadísticas de los nodos recorridos.

Un aspecto relevante del diseño consiste en que el MCTS no opera de manera ciega, sino que reutiliza conocimiento heurístico del dominio para ordenar movimientos, favorecer jugadas estructuralmente razonables y guiar las simulaciones. En consecuencia, el agente final combina la capacidad táctica del enfoque heurístico con la exploración probabilística propia de MCTS, obteniendo así una estrategia más sólida que cualquiera de los dos enfoques utilizados de manera aislada.